Hardware Acceleration

HW/SW Codesign - Week 12

Today's Topics

- Vector instructions and SIMD
- Optimized kernels
- NPUs
- Exploiting hardware acceleration

Three Levels of Acceleration on Microcontrollers

- No acceleration
  - Sequential execution of scalar arithmetic
- Vector instructions
  - Parallel execution of small vector arithmetic
- Neural Processing Unit (NPU)
  - Coprocessor that executes entire layers or models

No Acceleration

- Sequential execution of scalar arithmetic
- Can be written in standard C/C++

No Acceleration: 2D Convolution

- 2D convolution layer with valid padding and strides (1, 1):

$$a_{i,j,k} = \phi\left(b_k + \sum_{u=0}^{H-1}\sum_{v=0}^{W-1}\sum_{c=0}^{C-1} W_{k,u,v,c} \cdot x_{i+u,j+v,c}\right)$$

- Baseline C++ implementation requires:
  - 6 nested loops over *i*, *j*, *k*, *u*, *v*, and *c*
  - Multiply-accumulate (MAC) inside the innermost loop:
    - $acc = acc + W_{k,u,v,c} \cdot x_{i+u,j+v,c}$

No Acceleration: 2D Convolution in C++

```
void conv2d_s8_valid_stride11(const int8_t *w, const int8_t *x, const int output_h, ...)
{
    // Loop over activation elements
    for (int i = 0; i < output_h; i++) {
        for (int j = 0; j < output_w; j++) {
            for (int k = 0; k < output_ch; k++) {
                // Compute convolution
                int32_t acc = b[k];
                for (int u = 0; u < H; u++) {
                    for (int v = 0; v < W; v++) {
                        for (int c = 0; c < C; c++) {
                            int wi = ((k * H + u) * W + v) * C + c;
                            int xi = ((i + u) * input_w + j + v) * C + c;
                            acc += w[wi] * (x[xi] - zx);
                        }
                    }
                }

                // Rescale, shift, clip and store activation
                int32_t activation = min(max((acc * sa + za) >> 24, -128), 127);
                int ai = (i * output_w + j) * output_ch + k;
                a[ai] = static_cast<int8_t>(activation);
            }
        }
    }
}
```

No Acceleration: 2D Convolution in C++

```
void conv2d_s8_valid_stride11(const int8_t *w, const int8_t *x, const int output_h, ...)
{
    // Loop over activation elements
    for (int i = 0; i < output_h; i++) {
        for (int j = 0; j < output_w; j++) {
            for (int k = 0; k < output_ch; k++) {
                // Compute convolution
                int32_t acc = b[k];
                for (int u = 0; u < H; u++) {
                    for (int v = 0; v < W; v++) {
                        for (int c = 0; c < C; c++) {
                            int wi = ((k * H + u) * W + v) * C + c;
                            int xi = ((i + u) * input_w + j + v) * C + c;
                            acc += w[wi] * (x[xi] - zx);
                        }
                    }
                }

                // Rescale, shift, clip and store activation
                int32_t activation = min(max((acc * sa + za) >> 24, -128), 127);
                int ai = (i * output_w + j) * output_ch + k;
                a[ai] = static_cast<int8_t>(activation);
            }
        }
    }
}
```

No Acceleration: 2D Convolution in C++

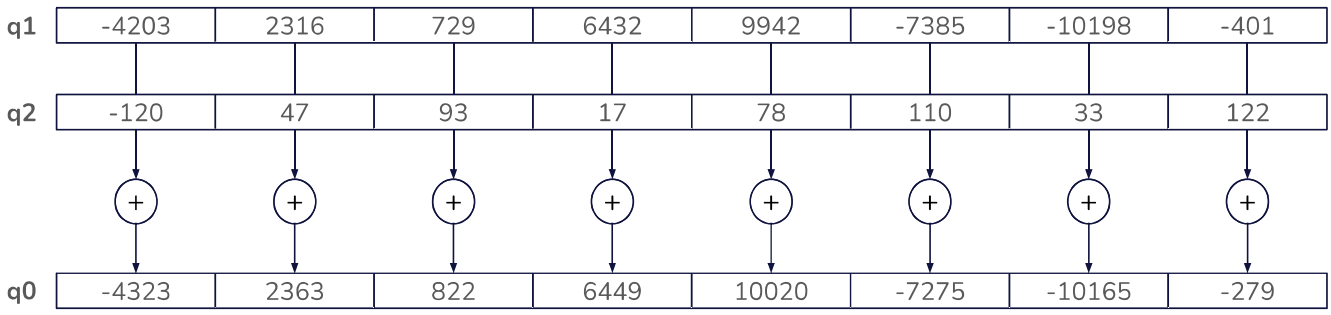
```
void conv2d_s8_valid_stride11(const int8_t *w, const int8_t *x, const int output_h, ...)
{
    // Loop over activation elements
    for (int i = 0; i < output_h; i++) {
        for (int j = 0; j < output_w; j++) {
            for (int k = 0; k < output_ch; k++) {
                // Compute convolution
                int32_t acc = b[k];
                for (int u = 0; u < H; u++) {
                    for (int v = 0; v < W; v++) {
                        for (int c = 0; c < C; c++) {
                            int wi = ((k * H + u) * W + v) * C + c;
                            int xi = ((i + u) * input_w + j + v) * C + c;
                            acc += w[wi] * (x[xi] - zx);
                        }
                    }
                }

                // Rescale, shift, clip and store activation
                int32_t activation = min(max((acc * sa + za) >> 24, -128), 127);
                int ai = (i * output_w + j) * output_ch + k;
                a[ai] = static_cast<int8_t>(activation);
            }
        }
    }
}
```

Vector Instructions

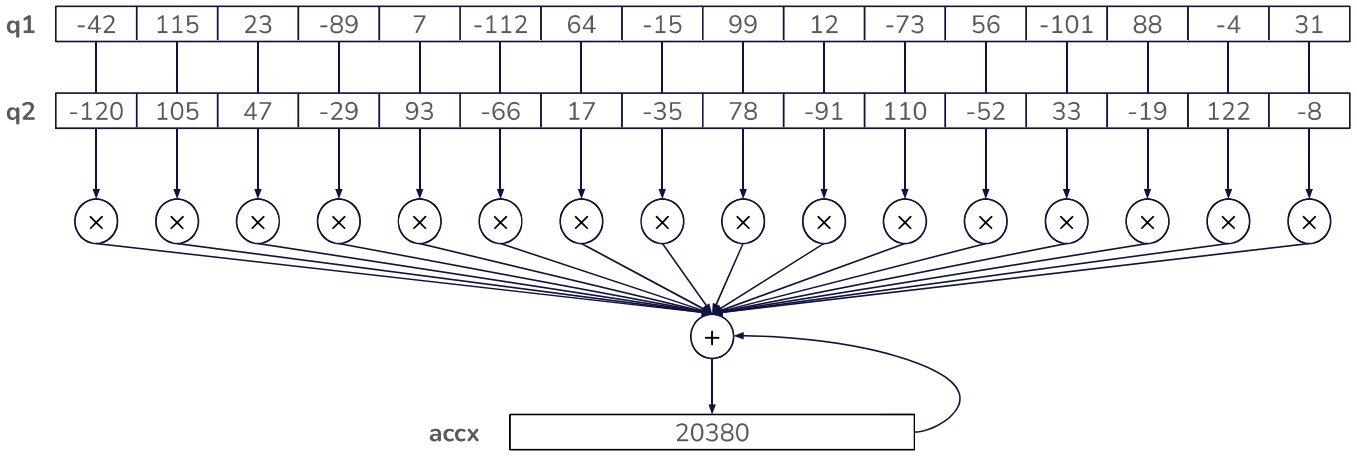
- Parallel execution of small vector arithmetic
- SIMD: Single instruction, multiple data
  - Multiple small elements side-by-side in wide registers

Vector Instructions

- Parallel execution of small vector arithmetic
- SIMD: Single instruction, multiple data
  - Multiple small elements side-by-side in wide registers
- Example: 8x16-bit addition in 128-bit registers



Vector Instructions: 16x8-bit MAC



Kernels

- Low-level implementation of a neural network operator
- Examples:
  - Dot product
  - Matrix multiplication
  - Fully connected layer
  - 2D convolution layer with filter size (3, 3), same padding and strides (1, 1)
  - Depthwise convolution
  - Activation functions, e.g. sigmoid, softmax, Swish
- Typically optimized for specific hardware
  - Example: ESP-NN (under managed_components in your project code)
  - Example: CMSIS-NN for ARM Cortex-M MCUs

Example: Optimized Kernel for ESP32-S3

```
void conv2d_s8(const ConvParams& p) {
    const bool fast_path = p.filter_weights % 16 == 0 && p.input_ch % 16 == 0;
    if (fast_path) {
        // Assembly fast path for ESP32-S3
        conv_s8_aligned_padded_esp32s3_asm(p);
    } else {
        // Slower but more general C implementation
        conv_s8_reference(p);
    }
}
```

Example: Optimized Kernel for ESP32-S3

```
void conv2d_s8(const ConvParams& p) {
    const bool fast_path = p.filter_weights % 16 == 0 && p.input_ch % 16 == 0;
    if (fast_path) {
        // Assembly fast path for ESP32-S3
        conv_s8_aligned_padded_esp32s3_asm(p);
    } else {
        // Slower but more general C implementation
        conv_s8_reference(p);
    }
}

conv_s8_aligned_padded_esp32s3_asm:
    ...                         # Load registers from ConvParams
.outer_loop:
    ee.zero.accx                # ACCX = 0 for this output pixel
.inner_loop:
    ee.vld.128.ip q1, a5, 16    # q1 = Next 16 signed 8-bit filter weights; a5 += 16
    ee.vld.128.ip q2, a13, 16   # q2 = Next 16 signed 8-bit input values; a13 += 16
    ee.vmulas.s8.accx q1, q2    # Vector MAC: ACCX += dot(q1, q2)
    bne a5,a9,.inner_loop       # Keep going until filter pointer reaches end
    rur.accx_0 a10              # a10 = ACCX[31:0]
    ...                         # Scale, clip, store, advance, branch to outer_loop
```

Example: Optimized Kernel for ESP32-S3

```
void conv2d_s8(const ConvParams& p) {
    const bool fast_path = p.filter_weights % 16 == 0 && p.input_ch % 16 == 0;
    if (fast_path) {
        // Assembly fast path for ESP32-S3
        conv_s8_aligned_padded_esp32s3_asm(p);
    } else {
        // Slower but more general C implementation
        conv_s8_reference(p);
    }
}
```

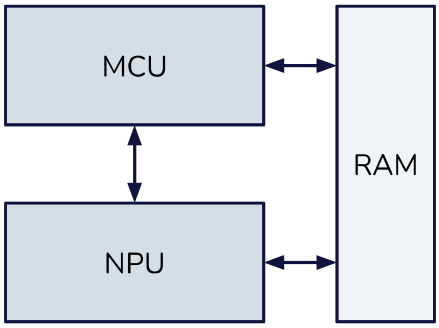
```
conv_s8_aligned_padded_esp32s3_asm:
    ...                         # Load registers from ConvParams
.outer_loop:
    ee.zero.accx                # ACCX = 0 for this output pixel
.inner_loop:
    ee.vld.128.ip q1, a5, 16    # q1 = Next 16 signed 8-bit filter weights; a5 += 16
    ee.vld.128.ip q2, a13, 16   # q2 = Next 16 signed 8-bit input values; a13 += 16
    ee.vmulas.s8.accx q1, q2    # Vector MAC: ACCX += dot(q1, q2)
    bne a5,a9,.inner_loop       # Keep going until filter pointer reaches end
    rur.accx_0 a10              # a10 = ACCX[31:0]
    ...                         # Scale, clip, store, advance, branch to outer_loop
```

ESP-NN Kernel Efficiency Gains

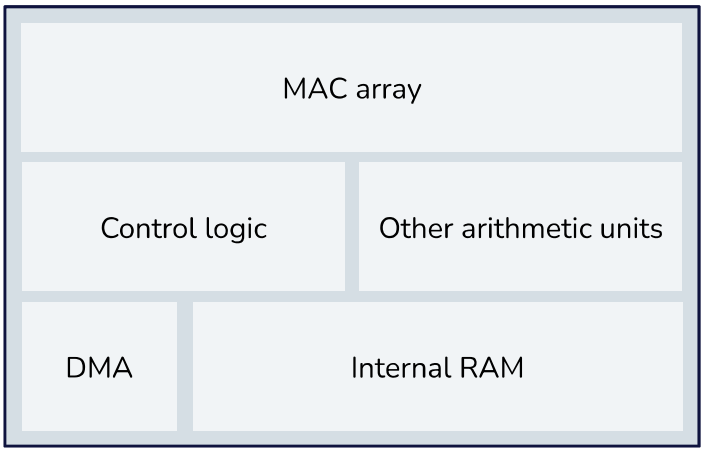
Function	ANSI C	Optimized	Cpt Ratio	Data info
elementwise_add	281337	74440	3.78	size = 1615
elementwise_mul	122703	35002	3.51	size = 1615
convolution	4712500	331008	14.24	input(10,10), filter(64x1x1x64), pad(0,0), stride(1,1)
convolution	312754	39022	8.01	input(8,8), filter(16x1x1x16), pad(0,0), stride(1,1)
convolution	2193289	394842	5.55	input(8,8), filter(64x3x3x3), pad(0,0), stride(1,1)
depthwise conv	1159831	184176	6.30	input(18,18), pad(0,0), stride(1,1), filter: 1x3x3x16
depthwise conv	1671363	372435	4.49	input(12,12), pad(1,1), stride(1,1), filter: 8x5x5x4
max pool	376294	48069	7.83	input(16,16), filter(1x3x3x16)
avg pool	427293	118052	3.62	input(16,16), filter(1x3x3x16)
fully connected	8443	1078	7.83	len: 271, ch = 3
softmax	15209	11107	1.37	h: 8, w: 32
prelu (relu6)	1125	98	11.48	size: 1615

Neural Processing Units (NPUs)

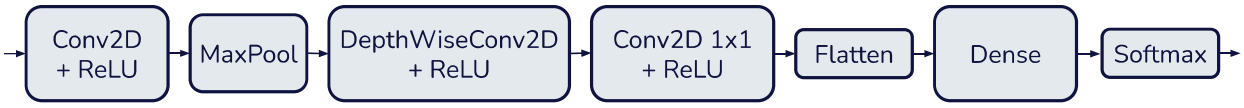
- Separate block on SoC
- Complete kernels in hardware
- Executes single layers, subgraphs or entire models



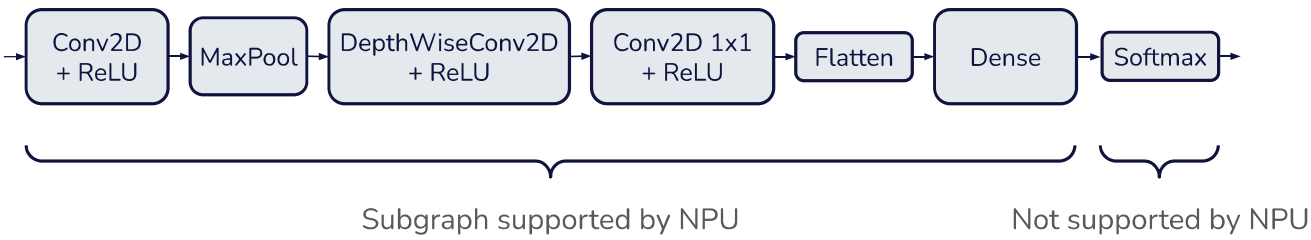
Neural Processing Units (NPUs)



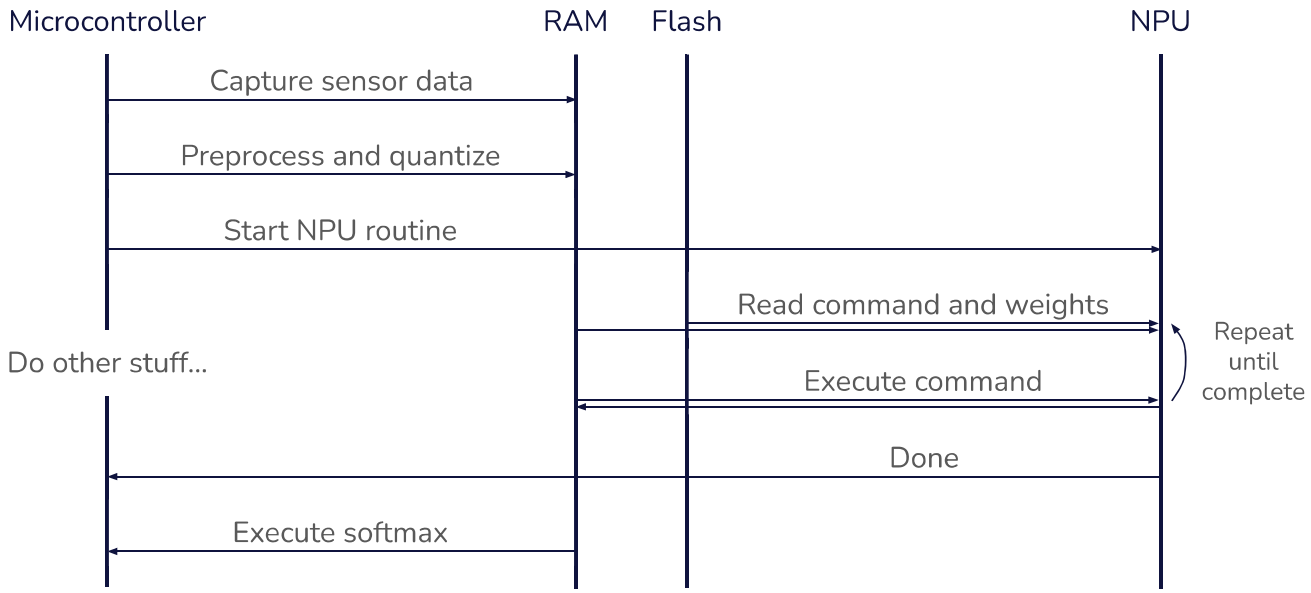
Example: CNN inference with NPU



Example: CNN inference with NPU



Example: CNN inference with NPU



ML Model Compilation for MCU and NPU

- Runs after training and optimization
- Compilation process:
  - Graph partitioning
  - Schedule:
    - Microcontroller computations
    - NPU computations
    - Memory transfers
  - Produce binary with:
    - Computation graph for microcontroller
    - Command streams for NPU

ML Model Compilation for MCU and NPU

- Runs after training and optimization
- Compilation process:
  - Graph partitioning
  - Schedule:
    - Microcontroller computations
    - NPU computations
    - Memory transfers
  - Produce binary with:
    - Computation graph for microcontroller
    - Command streams for NPU

ML Model Compilation for MCU and NPU

- Computation graph for microcontroller:

- Command stream for NPU:

- `LoadInput` (*address*, *length*)
- `Conv2D+ReLU` (*conv2d parameters*)
- `MaxPool` (*maxpool parameters*)
- `DepthwiseConv2D+ReLU` (*depthwiseconv2d parameters*)
- `Conv2D+ReLU` (*conv2d parameters*)
- `Reshape` (*reshape parameters*)
- `Dense` (*dense parameters*)
- `StoreOutput` (*address*, *length*)

NPU Example: ARM Ethos-U

HIGHLIGHTS

- Energy Efficient

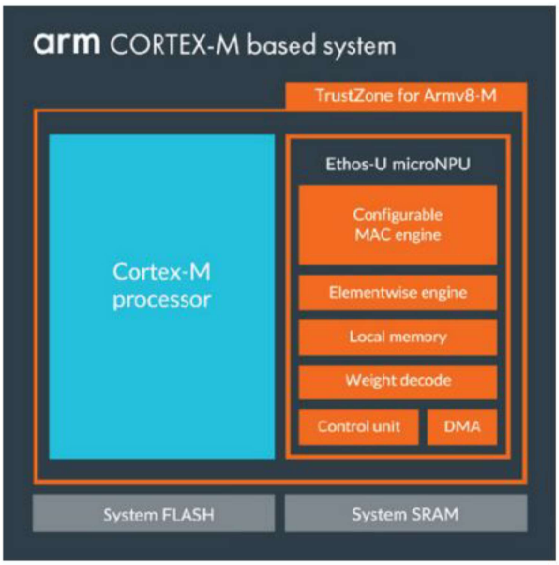
  Provides up to 90% energy reduction for ML workloads, such as ASR, compared to previous Cortex-M generations.

- Network Support

  Flexible design supports a variety of popular neural networks, including CNNs and RNNs, for aucio processing, speech recognition, image classification, and object detection.

- Future-Proof Operator Coverage

  Heavy compute operators run directly on the NPU, such as convolution, transformer, LSTM, RNN, pooling, activation functions, and primitive element-wise functions. Other kernels can run automatically on the tightly coupled Cortex-M using CMSIS-NN or Arm Compute Library on Cortex-A.



NPU Example: ARM Ethos-U

HIGHLIGHTS

- Energy Efficient

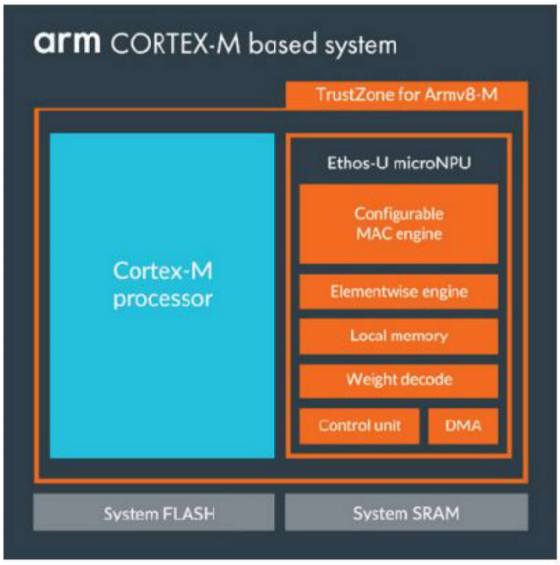
  Provides up to 90% energy reduction for ML workloads, such as ASR, compared to previous Cortex-M generations.

- Network Support

  Flexible design supports a variety of popular neural networks, including CNNs and RNNs, for audio processing, speech recognition, image classification, and object detection.

- Future-Proof Operator Coverage

  Heavy compute operators run directly on the NPU, such as convolution, transformer, LSTM, RNN, pooling, activation functions, and primitive element-wise functions. Other kernels can run automatically on the tightly coupled Cortex-M using CMSIS-NN or Arm Compute Library on Cortex-A.



Exploiting Hardware Acceleration

Where machine learning finally meets codesign

Exploiting Hardware Acceleration: Three Design Cases

1. Fixed hardware
    - Design and optimize the model for the given hardware
2. Selectable hardware
    - Design model first, choose fitting hardware that supports design
3. Custom silicon
    - Codesign model and hardware

1. Fixed Hardware

Design and optimize the model for the given hardware:

- Memory
  - Example: 512 kB SRAM ⇒ Keep peak RAM usage below 512 kB
- Optimized kernels
  - Example: Optimized kernel for depthwise convolution with:
    - Filter size 3x3
    - Same padding
    - Number of input channels is multiple of 16
- Hardware blocks
  - Example: FPU ⇒ Consider quantizing to FP8 or FP16
  - Example: NPU available ⇒ Prefer layers supported by NPU

2. Selectable Hardware

Design model first, choose fitting hardware that supports design:

- Memory
  - Example: Peak RAM usage 768 kB ⇒ Use MCU with at least 1MB SRAM
- Optimized kernels
  - Example: Model uses depthwise convolution ⇒ Ensure optimized kernels
- Hardware blocks
  - Example: Model quantizes badly to integers ⇒ Ensure FPU

Codesign Mapping Problems (From Lecture 1)

Mapping problems

- Resource allocation
  - Processing units, memories, interconnect, …
  - Some resources may need to be synthesized in design flow
- Binding
  - Map functionality to processing units, data to memories, communication to routes
- Scheduling
  - Order of execution, priority scheduler, or similar for each processing unit, communication and memory resources
  - Relative priorities of tasks bound to a processor, etc.

3. Custom Silicon

Jointly codesign model, allocations, bindings and scheduling:

- **Model**: The full model and optimization design space
- **Allocations**, for example:
  - Processor cores
  - Hardware kernels
  - RAM and flash
- **Bindings**, for example:
  - Weights ⇒ RAM or flash?
  - Softmax ⇒ Hardware kernel or processor core?
- **Scheduling**, for example:
  - Schedule weight transfer vs layer execution

Next Week

- Summary
- Q&A
- Project work
- 23:59: Project submission deadline
- Preparations:
  - Test exam
  - Prepare questions for Q&A
  - Finish the project!